Genetic Programming Hyper-Heuristics for the Job Shop Scheduling Problem with Interval Durations: Interpretable Rules that Generalize Across Instance Sizes

Hernán Díaz

Abstract

Dispatching rules remain the method of choice for scheduling under tight time budgets, but hand-crafted rules leave substantial performance unexploited, and the automated design of rules has scarcely been explored for scheduling problems with non-deterministic processing times. We present a genetic programming (GP) hyper-heuristic for the job shop scheduling problem with interval durations (IJSP), in which operation durations are known only to lie within an interval and schedules are evaluated by interval arithmetic. The GP evolves priority expressions over an *interval-aware* terminal set that exposes both the worst-case values and the widths of the uncertain quantities, so that evolved rules can react to uncertainty itself. On the 70 interval versions of Taillard's benchmark, the best evolved rule attains a mean relative error of 18.5% with a single deterministic rollout of negligible cost—improving on the strongest deterministic constructive baseline (Giffler–Thompson with MWKR tie-breaking, 29.4%) by over ten points—and generalizes zero-shot across all seven instance size classes despite being evolved on a single class. Embedded in a GRASP-style randomized scheme, the evolved rule converts sampling budget into solution quality (14.1% at best-of-1024). Evolved rules are compact, human-readable expressions, and an automatic configuration study with irace shows the approach is robust to its own hyperparameters, with a preference for *smaller* expression trees at no performance cost. We position the method against fixed rules, active schedule generators, and published metaheuristics, characterizing the computational class in which interpretable evolved rules operate.

1 Introduction

Real manufacturing environments rarely know the exact duration of an operation in advance. The interval job shop scheduling problem (IJSP) models this uncertainty in a minimal, distribution-free way: each processing time is only assumed to lie within a known interval, schedules are built and evaluated with interval arithmetic, and solutions are compared by their worst-case (upper-bound) makespan. Population metaheuristics for the IJSP—artificial bee colony variants and tabu search among them—achieve strong results at the cost of minutes of per-instance search [1, 2, 3].

At the opposite end of the computational spectrum live *dispatching rules*: priority functions that build a schedule in milliseconds by repeatedly selecting the next operation to start. Their appeal is threefold—speed, interpretability, and trivial deployment—but hand-crafted rules (SPT, MOR, MWKR and relatives) leave a large quality gap: on the benchmark used in this paper the best fixed rule averages 45.4% relative error, versus 4–10% for metaheuristics.

For *deterministic* scheduling, a rich literature has shown that this gap can be narrowed substantially by evolving rules automatically with genetic programming (GP) [5, 6]. For scheduling under interval uncertainty, however, the automated design of dispatching rules remains, to the best

of our knowledge, unexplored. It is not obvious a priori what an evolved rule should do with uncertainty: ignore it and rank by worst case, hedge against wide intervals, or exploit width information in a structured way. Making this possible requires exposing uncertainty to the rule as first-class attributes.

This paper makes the following contributions:

1. A GP hyper-heuristic for the IJSP whose terminal set is *interval-aware*: alongside worst-case processing time, earliest start and work remaining, rules can read the *widths* of these quantities, enabling uncertainty-sensitive prioritization (Section 4).

2. An empirical study on the 70 interval Taillard instances showing that evolved rules (i) beat every deterministic constructive baseline by a wide margin at equal cost, (ii) generalize zero-shot across instance sizes—evolved on 20×15, best class 50×15—and (iii) extend naturally to a randomized multi-start regime with monotone quality gains up to best-of-1024 (Section 6).

3. An automatic configuration study (irace, 200 experiments) of the evolutionary hyperparameters, showing the method is robust to its own knobs: tuning yields only a modest, within-noise gain, from higher selection pressure (Section 6.5).

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 formalizes the IJSP. Section 4 presents the GP hyper-heuristic and its interval-aware terminal set. Section 5 details the experimental protocol, and Section 6 the empirical study, including the hyperparameter analysis. Section 7 concludes.

2 Related Work

2.1 Job shop scheduling under interval uncertainty

Interval durations offer a distribution-free uncertainty model requiring only bounds, in contrast to stochastic and fuzzy approaches [11]. Prior IJSP work is dominated by population-based metaheuristics: an elitist artificial bee colony with interval ranking [1], its ensemble-ranking successor [2], and a population-based tabu search that currently defines the state of the art [3]. All operate in the minutes-per-instance regime. Fast constructive methods for the IJSP have received far less attention; dispatching rules with interval-aware lexicographic comparisons are the de facto baseline.

2.2 Automated design of dispatching rules

GP hyper-heuristics evolve priority expressions over scheduling attributes, and constitute the standard approach to automated rule design for production scheduling [5, 6]. Evolved rules have been shown to outperform hand-crafted ones across job shop variants, with active research on surrogate fitness, feature selection, and multi-objective formulations. Closest to our setting, Karunakaran et al. [7] evolve rules for dynamic job shops with uncertain processing times, exposing summary statistics of observed deviations (an exponential moving average) to the rule; uncertainty there is realized stochastically during simulation, and rules are evaluated on sampled scenarios. A recent survey of learned optimisation for the job shop [8], covering both GP and DRL constructive approaches, records no work on interval-valued processing times. To the best of our knowledge, evolving rules whose *inputs* are interval attributes—and whose fitness is computed by interval arithmetic rather than by sampling realizations—has not been explored before.

2.3 Learned constructive heuristics

Deep reinforcement learning provides an alternative family of learned constructive methods for the JSP [9, 10]; GP rules and DRL policies are increasingly studied as competing learning paradigms for the same construction task, and controlled comparisons between them under common protocols have been identified as a gap in the recent literature [8]. Both families occupy the same computational class at deployment (milliseconds per decision); a controlled comparison on the IJSP is beyond the scope of this paper and is the object of ongoing work.

3 The Interval Job Shop Scheduling Problem

An IJSP instance consists of n jobs and m machines. Job j is a chain of m operations $o_{j1} \prec o_{j2} \prec \cdots \prec o_{jm}$, each requiring a distinct machine $\mu(o_{jk})$ exclusively and non-preemptively. The duration of operation o is an interval $p_o = [\underline{p}_o, \overline{p}_o]$ with $0 < \underline{p}_o \leq \overline{p}_o$: the only assumption is that the realized duration lies within these bounds.

Schedule construction uses interval arithmetic. For intervals $a = [\underline{a}, \overline{a}]$ and $b = [\underline{b}, \overline{b}]$: $a + b = [\underline{a} + \underline{b},\ \overline{a} + \overline{b}]$ and $\max(a, b) = [\max(\underline{a}, \underline{b}),\ \max(\overline{a}, \overline{b})]$ (component-wise). Given an operation processing order (a permutation with repetition of jobs, decoded left to right), the *semiactive* schedule starts each operation at $s_o = \max(C_{\text{job}}, C_{\text{mach}})$—the component-wise maximum of the completion intervals of its job predecessor and machine predecessor—and completes at $C_o = s_o + p_o$. The makespan $C_{\max} = \max_j C_{o_{jm}}$ is itself an interval; following the IJSP literature [4, 2], schedules are ranked lexicographically by upper bound (worst case) first. All methods in this paper—the GP rules and every baseline—share one implementation of this decoder and evaluator, so reported differences cannot stem from evaluator discrepancies.

Performance is reported as relative error against the crisp Taillard lower bounds: $\text{RE} = (E[C_{\max}] - LB)/LB \times 100$, where $E[C_{\max}]$ is the midpoint of the interval makespan.

4 GP Hyper-Heuristic for the IJSP

4.1 Rule representation

An individual is a priority expression tree. Leaves are drawn from the interval-aware terminal set of Table 1; internal nodes from $\{+, -, \times, \div_p, \min, \max, \text{neg}\}$ with protected division. At each scheduling decision the tree is evaluated (vectorized) over all eligible operations and the operation with *lowest* priority is dispatched, yielding a semiactive schedule. Classic rules are points of this search space—SPT is the tree PT and MWKR is $\text{neg}(WKR)$—and both are injected as seed individuals.

Protected division is defined as $a \div_p b = a/b$ if $|b| > 10^{-9}$ and a otherwise. Priority ties at dispatch time are broken by eligible set order (lowest job index first), which is deterministic; hence a rule maps each instance to exactly one schedule.

4.2 Evolution

Generational GP with the operators and parameters of Table 2. Initialization is ramped half-and-half over depths 2–4 (grow trees terminate early with probability 0.3 per node), with SPT and MWKR injected as two additional seed individuals. Each generation, the elite passes unchanged and the remainder is produced by tournament selection followed by subtree crossover (a uniformly random node of parent A is replaced by a uniformly random subtree of parent B) or, with the

Table 1: Interval-aware terminal set, with exact definitions. For an eligible operation o of job j on machine m: $p_o = [\underline{p}_o, \overline{p}_o]$ is its duration interval; C_j and C_m are the (interval) completion times of the last scheduled operation of job j and machine m; $\mathcal{R}_o$ is the set of operations of j after o; $\mathcal{E}$ is the current eligible set. Terminals are evaluated on *raw* (unnormalized) values.

Terminal	Definition	Role
PT	$\overline{p}_o$	worst-case processing time
PTW	$\overline{p}_o - \underline{p}_o$	duration uncertainty
EST	$\overline{e}_o = \max(\overline{C}_j, \overline{C}_m)$	worst-case earliest start
ESTW	$\overline{e}_o - \underline{e}_o$, with $\underline{e}_o = \max(\underline{C}_j, \underline{C}_m)$	start uncertainty
WKR	$\sum_{q \in \mathcal{R}_o} \overline{p}_q$	worst-case work remaining
WKRW	$\sum_{q \in \mathcal{R}_o} (\overline{p}_q - \underline{p}_q)$	remaining-work uncertainty
NOR	$\lvert\mathcal{R}_o\rvert$	operations remaining
SLACK	$\overline{e}_o - \min_{o' \in \mathcal{E}} \overline{e}_{o'}$	start slack within eligible set
ONE	1	constant

Table 2: Evolution parameters (conventional settings; see Section 6.5 for the sensitivity study).

Population / generations	100 / 50
Initialization	ramped half-and-half, depths 2–4 + {SPT, MWKR}
Selection	tournament, size 4
Crossover / mutation prob.	0.8 / 0.2 (subtree; mutation depth ≤ 3)
Elitism	2
Tree-size cap	40 nodes (violations: fitness ∞)
Fitness	mean RE, deterministic rollout, TA11–TA14
Seeds	$\{1, 2, 3\}$ (RNG of evolution and environment)

complementary probability, subtree mutation (a uniformly random node is replaced by a fresh grow tree of depth ≤ 3). Offspring exceeding the node cap receive fitness ∞ (bloat control). Fitness of a rule is the mean RE of its deterministic rollout over the four training instances—computed with the same environment, decoder and evaluator used by all baselines, which eliminates evaluator mismatch as a confound.

4.3 Randomized multi-start extension

A deterministic rule produces one schedule. For matched comparisons with sampling-based methods we wrap the evolved rule in GRASP-style randomization: with probability ε=0.1 the dispatched operation is drawn uniformly from the eligible set, otherwise the rule decides; sample 0 remains deterministic. Best-of-N then reports the lexicographically best schedule.

5 Experimental Setup

Instances. The 70 interval versions of Taillard's instances (TA1–TA70, seven size classes from 15×15 to 50×20), with crisp Taillard lower bounds as the RE reference. Each interval instance is derived from its crisp counterpart by replacing every duration p with an integer interval $[p-d,\ p+d]$ symmetric around it, with half-width bounded by $d \leq 0.15\,p$ (maximum deviation 15%); machine routes are unchanged. The midpoint of every interval thus equals the original duration—the crisp instance is exactly the midpoint scenario of its interval version—which makes the crisp Taillard

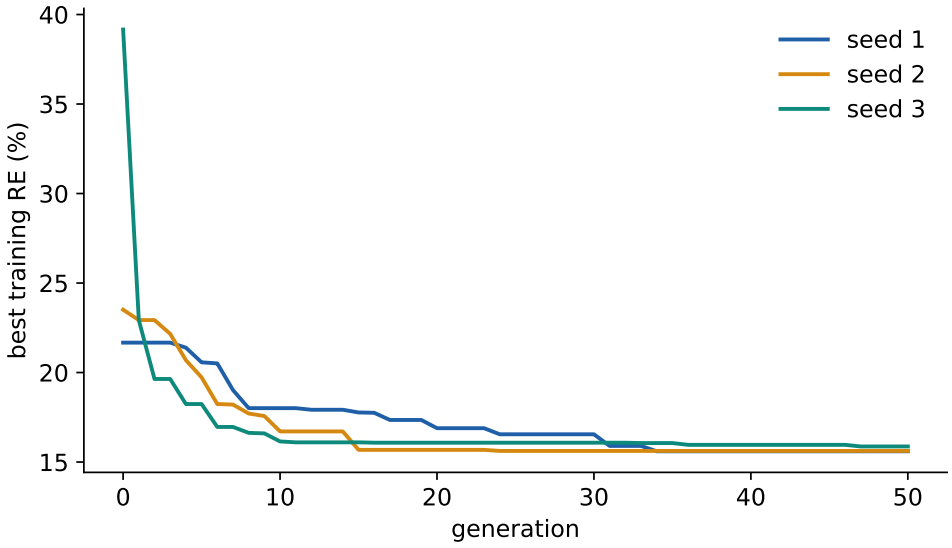

Figure 1: Best training fitness per generation for the three evolution seeds (default configuration).

bounds a meaningful reference for $E[C_{\max}]$. After integer rounding, ≈20% of operations end up degenerate (deterministic), and the mean relative width is ≈13%. This is the same ±15% generation scheme used throughout our prior work on the IJSP [1, 2, 3], which makes the comparison against those published metaheuristics an exact one (identical uncertainty model and benchmark).

Data split and exclusions. Rules are evolved on TA11–TA14 (20×15), and TA15–TA20 serve as development set: they never contribute to fitness, but design and configuration decisions (including the irace study of Section 6.5) were selected on their performance, so we flag them as subject to development reuse. The remaining 60 instances are fully unseen by every design decision and constitute the honest generalization test.

Budgets. Population 100, 50 generations: ∼20,400 fitness rollouts (≈49 min single-core) per seed; three independent seeds.

Baselines. Fixed rules (SPT, LPT, MOR, MWKR) with interval-aware lexicographic comparison; the Giffler–Thompson active-schedule generator with rule tie-breaks (deterministic and ε-randomized inside the conflict set); published IJSP metaheuristics (fEABC [1], ESABC [2], TS [3]) as yardsticks of the search-based computational class.

6 Results

6.1 Evolution behaviour and seed stability

Three seeds converge to training RE of 15.60%, 15.62% and 15.87%—a spread of 0.27 points—with most of the improvement realized by generation 25–30 (Figure 1). The best evolved rules are structurally recognizable: MWKR corrected by earliest-start and slack×processing-time terms. For example, the best rule of seed 1 is, after simplification, of the form $-\big(WKR - f(NOR, PT, PTW)\big) + EST + SLACK \cdot PT$: maximize work remaining, penalize late earliest starts, and penalize slack weighted by processing time.

Table 3: Mean RE (%) per size class, single deterministic rollout, three GP seeds; all 70 Taillard interval instances.

Rule	15×15	20×15	20×20	30×15	30×20	50×15	50×20	All
GP seed 1	16.0	18.7	19.4	20.9	25.0	**14.5**	15.1	**18.5**
GP seed 2	16.5	18.4	21.2	21.8	26.9	15.1	16.3	19.5
GP seed 3	18.6	18.5	20.0	22.7	25.7	15.2	15.7	19.5

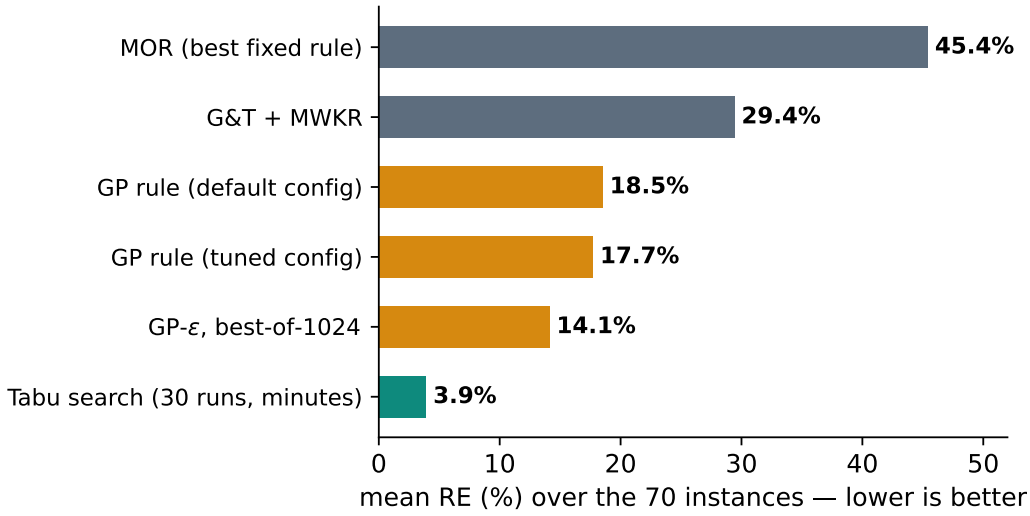

Figure 2: Constructive-method ladder on the 70 interval Taillard instances. Bars are single-solution methods except where noted.

6.2 Generalization to the full benchmark

Table 3 reports single-rollout RE per size class over all 70 instances. The best rule attains 18.5% global mean—evolved only on 20×15, its best class is the unseen 50×15 (14.5%). Because all terminals are per-operation attributes with no size-bound quantities, transfer requires no retraining or rescaling.

6.3 Position among constructive methods

At equal cost (one deterministic rollout): MOR 45.4%, G&T-MWKR 29.4%, evolved rule 18.5% (tuned configuration: 17.7%, Section 6.5). The evolved rule improves on the strongest deterministic constructive baseline by 10.9 points while remaining a closed-form, human-readable expression. Figure 2 places these results on the full method ladder, including the published tabu search [3] as the yardstick of the search-based computational class (minutes per instance versus milliseconds).

6.4 Sampling behaviour (GP-ε)

Best-of-N over all 70 instances: 18.5% (N=1, deterministic), 17.1% (16), 15.9% (64), 14.9% (256), 14.1% (1024)——4.4 points over ten doublings (Figure 3)—closing a substantial part of the gap to population-based methods while retaining millisecond-class construction.

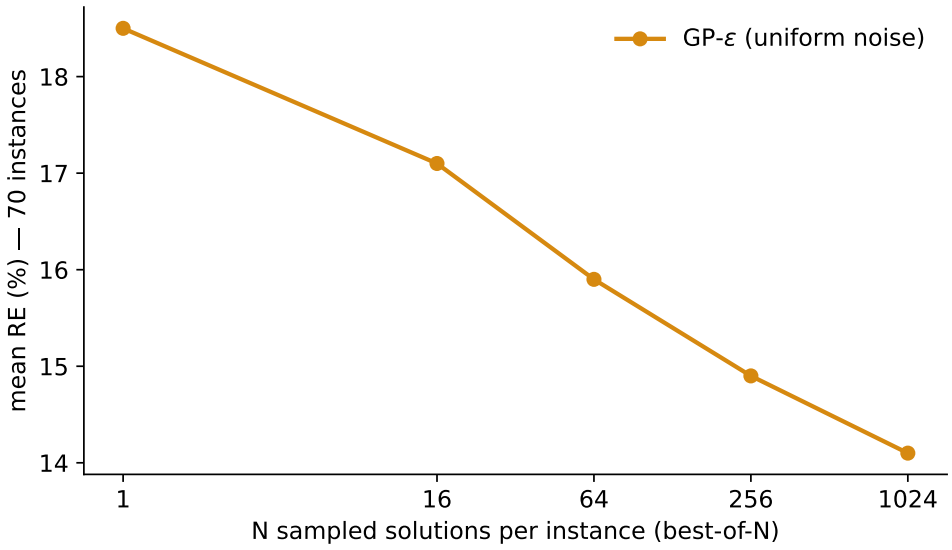

Figure 3: Best-of-N curve of the evolved rule with uniform ε-randomization (ε=0.1): mean RE over the 70 instances as the sampling budget doubles.

6.5 Hyperparameter study

We tune the four qualitative knobs of the evolution (tournament size, crossover probability, tree-size cap, elitism) with irace over a budget of 200 experiments, each a full evolution (population 100, 50 generations) on the training instances evaluated on the development set, with the conventional configuration seeded. The four elite configurations agree on a higher selection pressure than the default (tournament size 7 versus 4) and a crossover probability close to the default (0.72–0.77 versus 0.8); the tree-size cap, by contrast, does not discriminate (surviving elites span caps of 20, 30 and 40), indicating the method is insensitive to bloat control within this range. Throughout the race the statistical tests discard configurations only weakly, itself evidence that performance is largely flat over the hyperparameter space.

Under the same pre-registered confirmation applied to every design decision—the winning configuration re-evolved over three independent seeds and each resulting rule evaluated on all 70 instances—the tuned configuration is *marginally* better than the untuned default: 17.67% versus 18.5% best-of-three-seeds (18.68% versus 19.17% mean). The improvement of 0.5–0.8 points is consistent in direction across both statistics but lies within the seed-to-seed variance ($\approx$2 points), so we report it as a modest gain rather than a decisive one, and characterize the method as robust to its hyperparameters with a mild benefit from stronger selection pressure.

7 Conclusions and Future Work

We presented what is, to the best of our knowledge, the first GP hyper-heuristic for job shop scheduling with interval durations, built on a terminal set that exposes interval widths as first-class attributes. The empirical conclusions are three. (i) Evolved rules dominate every deterministic constructive baseline on the 70-instance benchmark by a wide margin (18.5% versus 29.4% for the strongest one) at negligible cost, and (ii) they generalize zero-shot across all size classes—evolved on 20×15, best on the unseen 50×15—because the representation contains no size-bound quantities.

(iii) The method is robust to its evolutionary hyperparameters: a 200-experiment irace study yields only a modest, within-noise gain over conventional settings, from higher selection pressure.

Future work follows four lines. First, a terminal ablation to quantify how much the evolved rules actually exploit the interval widths—the distinctive ingredient of this setting. Second, richer symbolic models: ensembles of complementary rules and co-evolution of construction and tie-breaking. Third, using evolved rules as cheap, high-quality population initializers for the metaheuristics that define the state of the art in the IJSP, where preliminary pools built from randomized rule rollouts are already compatible with the seeding protocol of [3]. Fourth, a controlled comparison with learned neural constructive policies under matched training and inference budgets—the head-to-head evaluation that the recent literature [8] identifies as missing.

Reproducibility

[TODO: URL/DOI del repositorio al publicar]

References

[1] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. Fast elitist ABC for makespan optimisation in interval JSP. *Natural Computing*, 22(4):645–657, 2023.

[2] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. An elitist seasonal artificial bee colony algorithm for the interval job shop. *Integrated Computer-Aided Engineering*, 30(3):223–242, 2023.

[3] H. Díaz et al. Neighbourhood structures and ranking operators for the interval job shop scheduling problem. Under review, 2026.

[4] D. Lei. Population-based neighborhood search for job shop scheduling with interval processing time. *Computers & Industrial Engineering*, 62(4):1200–1208, 2012.

[5] J. Branke, S. Nguyen, C. W. Pickardt, M. Zhang. Automated design of production scheduling heuristics: a review. *IEEE Transactions on Evolutionary Computation*, 20(1):110–124, 2016.

[6] S. Nguyen, Y. Mei, M. Zhang. Genetic programming for production scheduling: a survey with a unified framework. *Complex & Intelligent Systems*, 3(1):41–66, 2017.

[7] D. Karunakaran, Y. Mei, G. Chen, M. Zhang. Dynamic job shop scheduling under uncertainty using genetic programming. In *Intelligent and Evolutionary Systems*, pages 195–210. Springer, 2017.

[8] M. Xu, Y. Mei, F. Zhang, M. Zhang. Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning. *Artificial Intelligence Review*, 58:160, 2025.

[9] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, C. Xu. Learning to dispatch for job shop scheduling via deep reinforcement learning. *NeurIPS*, 2020.

[10] J. Park, J. Chun, S. H. Kim, Y. Kim, J. Park. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research*, 59(11):3360–3377, 2021.

[11] J. J. Palacios, I. González-Rodríguez, C. R. Vela, J. Puente. Coevolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop. *Fuzzy Sets and Systems*, 278:81–97, 2015.